

portfolio - frontend engineer

김동현

Kim Donghyun

복잡한 비즈니스를 **질서 있는 설계**로 바꾸고,
그 판단력으로 **AI를 통제**해 더 빠르고 안정적으로 만들어요.

기술로 비즈니스 프로세스를 개선하는 것이 개발자의 책무라고 믿어요.

akainool23@gmail.com

두 가지 역량으로 일해요

AI가 코드를 짜는 시대에 맞춰, '무엇이 옳은 결과인가'를 판단하고자 노력해 왔어요

01

비즈니스 이해를 기술로 연결하는 설계

- 비즈니스를 이해해야 무엇이 반복이고 무엇이 달라지는지 보여요
- 변하지 않는 것은 구조로, 변하는 것은 데이터로 분리해요
- 반복 업무는 사람이 아니라 구조가 흡수하게 만들어요
- 일회성 해결 대신, 다음 변화까지 감당하는 설계를 지향해요

02

AI를 통제하는 활용 능력

- AI는 가속기일 뿐, 품질의 수렴과 최종 책임은 개발자의 몫이에요
- 코드를 만드는 일은 흔해졌지만, 품질을 수렴시키는 일은 여전히 드물어요
- 개발 원칙과 규칙을 먼저 정의하고, AI가 그 안에서 움직이게 해요
- 산출물을 그대로 믿지 않고, 검증 게이트를 통과한 것만 믿어요

역량 AI를 통제하는 활용 능력

레거시 신탁 시스템 마이그레이션

Claude Code(Opus 4.6~4.8 + Fable) 위에 5단계 에이전트 파이프라인을 직접 설계했어요

INPUT 화면코드(예: CAW4800I) · 유실된 테이블 컬럼명 · 화면 패턴(6유형 중 택 1) — 기본 정보는 사람이 직접 넣어줘요



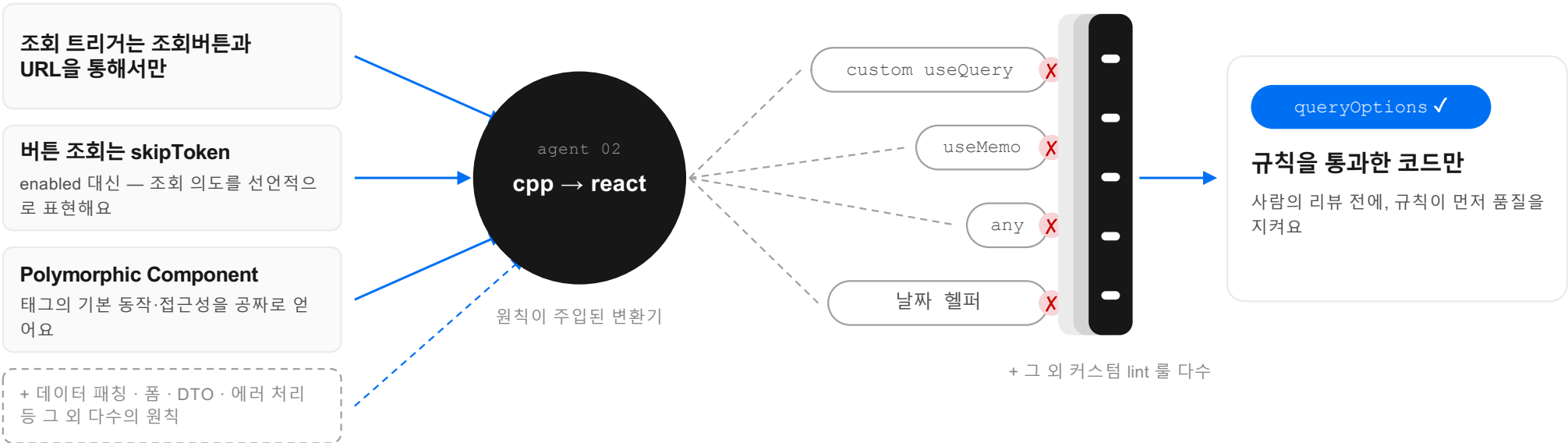
역량 AI를 통제하는 활용 능력

레거시 신탁 시스템 마이그레이션

원칙 문서 6개 계층(금지 143·필수 57) — 판단이 필요한 건 원칙으로, 기계가 잡을 건 lint로 나눴어요

inject - react principles

lint - mechanical gate



기계가 잡을 수 있는 건 lint로 내렸어요.

역량

AI를 통제하는 활용 능력

레거시 신탁 시스템 마이그레이션

Playwright Tester — 공통 ‘조회’의 3조건’에 화면 템플릿별 시나리오를 더해 검증해요

④ playwright tester 화면 템플릿(6유형)에 맞는 테스트 코드를 생성해, 통과할 때까지 검증해요

공통 — 조회의 3 요구사항

- ① 재조회 같은 조건을 다시 조회해도 로딩이 보이고, 실제로 재조회돼요
- ② URL 매핑 조회조건이 URL에 실려요 — 링크 공유가 곧 화면 공유
- ③ 자동 조회 조건이 실린 URL로 열면 즉시 조회돼요 — 뒤로가기도 기대대로

템플릿별 — 6유형 맞춤 시나리오

- CRUD 유형 신규 등록 → 업데이트 → 삭제를 Playwright가 직접 밟아요
- 조회 유형 최근 조회조건·결과 요약이 localStorage에 쌓여 화면에 보이는지 검사해요
- 특수 유형 사람이 지정한 패턴에 맞춰 테스트 코드를 새로 생성해요

그래도 확정 못 하는 것은 사람에게 — 예: WD_InitCombo는 두 API 중 무엇을 부르는지 원본만으론 알 수 없어요. 체크리스트 MD(documents/)에 기록되고, 사람이 확정해요

역량

비즈니스 이해를 기술로 연결하는 설계

레거시 신탁 시스템 마이그레이션

2~4줄을 차지하던 필터를 '점진적 공개'로 접었어요 — 클릭했을 때만 펼쳐져요

AS-IS - C++ 원본

[illegible]

TextInput·Select·DatePicker가 그대로 노출돼요 — 필터만 2~4줄을 차지했어요

필터 영역 4줄 → 1줄 — 필요할 때만 펼쳐요

TO-BE — React · FilterChip

CAW48001

판권자료 > 특정권접신작 > 조피관리 > 특정권도 수확잔액명세

이동목록

▼ 조피조건

조피 지역

기종일자 2026-05-22

원도분류

상품코드

원도코드

초기화

조회

I 기종일자

2026-05-22

결과 내 검색

Excel

기종일자	부점명	계좌번호	계약번호	위탁자구분	신규일자	만기일자	통화	수익잔액
5월 2026								
일	월	화	수	목	금	토		
					1	2		0.
3	4	5	6	7	8	9		0.
10	11	12	13	14	15	16		0.
17	18	19	20	21	22	23		0.
24	25	26	27	28	29	30		0.
31								0.
1300000A	성/비금신작14호	1048	광희문프리미...	001	2026-09-30	2025-10-14	KRW	0.
1300000E	정기예금신작14호	1048	광희문프리미...	001	2026-04-18	2026-04-17	KRW	0.
13000090	메리츠예금메우낚은위험TEST-001	1005	강남프리미어...	003	2025-06-27	2026-06-26	KRW	0.
1300009P	메리츠예금메우낚은위험TEST-002	1005	강남프리미어...	004	2025-07-15	2026-07-14	KRW	0.
1300009Q	메리츠예금메우낚은위험TEST-003	1005	강남프리미어...	006	2025-12-26	2026-12-28	KRW	0.
1300009R	메리츠예금메우낚은위험TEST-001	1005	강남프리미어...	008	2025-07-15	2026-07-14	KRW	0.
1400009	지사주신작_유한광형-001	1001	여의도리더스...	004	2025-09-30	2026-04-14	KRW	0.
			광희문프리미...	001	2026-09-30	2025-10-14	KRW	0.
			광희문프리미...	001	2026-10-02	2026-04-14	KRW	0.
			광희문프리미...	001	2026-10-10	2026-04-20	KRW	0.
			강남프리미어...	003	2025-12-18	2029-01-22	KRW	0.
			강남프리미어...	004	2025-12-19	2029-01-22	KRW	0.
			강남프리미어...	006	2025-12-22	2029-01-22	KRW	0.
			강남프리미어...	008	2025-12-22	2029-01-22	KRW	0.
			여의도리더스...	004	2025-09-24	2030-01-10	KRW	0.

전체 2228건

페이지당

전체 ▼

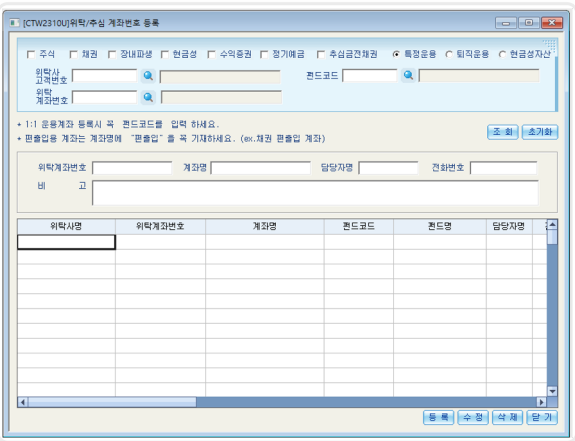
역량

비즈니스 이해를 기술로 연결하는 설계

레거시 신탁 시스템 마이그레이션

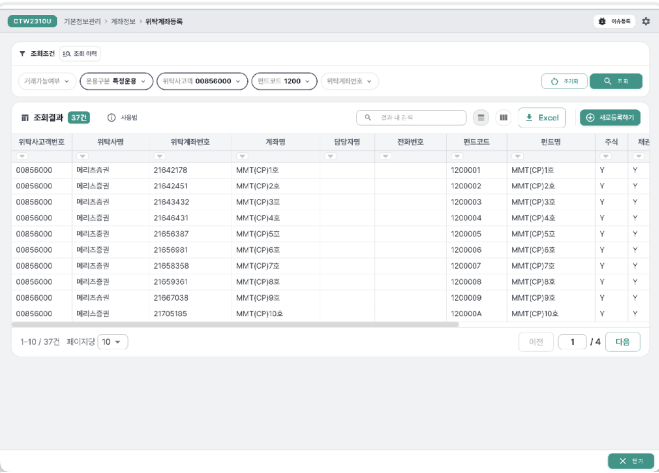
조회·등록·수정이 뒤섞여 있던 화면을 작업 단위로 분리했어요 — 조회는 메인, 편집은 Drawer로

AS-IS - C++ 원본



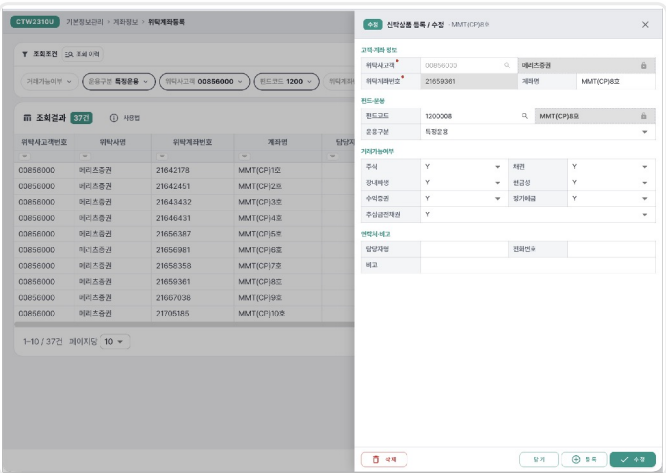
조회조건·등록/수정 인풋·조회 테이블이 한 화면에 — 지금 무슨 작업 중인지 화면이 안내하지 못했어요

TO-BE - InputDataGrid



메인은 조회조건+테이블만. 신규 등록은 헤더의 '새로등록하기' 버튼으로.

TO-BE - 행 클릭 시 Drawer



수정·삭제는 Drawer 레이어에서 — 작업의 경계가 명확해졌어요.

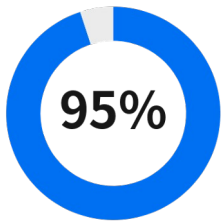
역량

AI를 통제하는 활용 능력

레거시 신탁 시스템 마이그레이션

결과 — 숫자로, 그리고 ‘더 나은 기능’으로 남았어요

result - in numbers



1차 개발자 QA를 통과했
어요

60여 개 이후 안정화됐고,
변환이후 개발자 QA에서 확인된 에러는 5% 미
만이에요

변환 초반

1h

화면당 변환 시간을 줄였
어요

파이프라인이 안정화된 뒤 3배 빨라
졌어요

파이프라인 이후

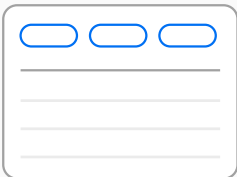
20m

품질은, 수치로 증명했어요.

better, not the same

같은 기능이 아닌, 더 나은 기능.

현대 웹 브라우저의 형태에 맞춰, 선제적으로 제시하고 적용했어요



스크롤 없는 정보 설계

업무를 직접 분석해, 좁은 팝업창 안에서도 스크롤 없이
정보 계층이 분리되게 재설계했어요

.../CAW4800I?fromDate=2026-01-02&fundCd=1200

웹 고유의 사용성 — URL 상태관리

조회조건이 URL에 실려요 — 링크를 공유하면 자동 조회되고, 뒤로가기도 기대대
로 동작해요

역량

비즈니스 이해를 기술로 연결하는 설계

설계, 세 가지 원칙으로 정리할 수 있어요

변하지 않는 것과 변하는 것을 나누는, 세 가지 기준이에요

01



변하지 않는 건 구조로, 변하는 건 데이터로

반복은 구조로 고정,
차이는 데이터로 분리해 SSOT를 만들어요

[거래처 관리](#)

[재무회계 통합](#)

[채용 SDUI](#)

[검사·설문
SDUI](#)

02



여러 선택지 중, 상황에 맞는 답을 골라요

일정·팀·운영 상황에 따라 최선이 달라요. 근거로 선택해요

[온보딩 튜토리얼](#)

[거래처 관리](#)

03



비즈니스 확장성과 코드 확장성을 함께 계산해요

확장이 예정됐다면 병목 전에 과감히 리팩토링해요

[재무회계 통합](#)

[검사·설문
SDUI](#)

[전역 리팩토링](#)

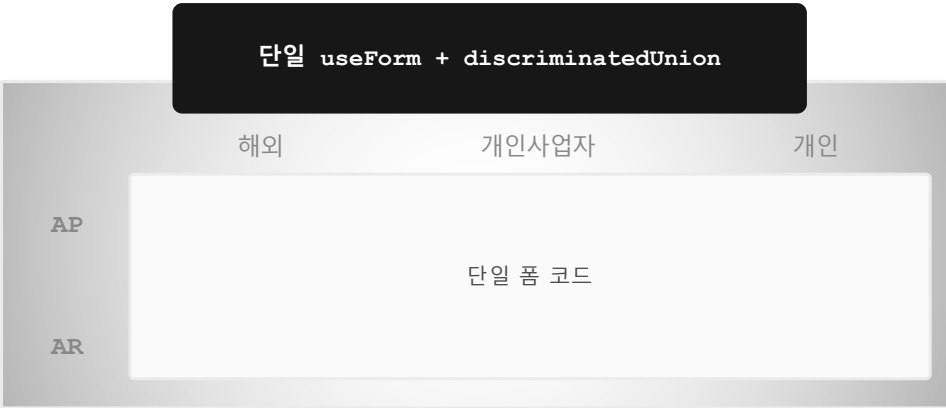
역량

비즈니스 이해를 기술로 연결하는 설계

거래처 관리 시스템 리뉴얼

타입마다 다른 6벌의 화면, 새로고침이면 사라지는 입력 — 두 문제를 하나의 설계로 풀었어요

다형 폼 — Zod Discriminated Union



- 6가지의 서로 다른 폼 구조를 zod 와 useForm 을 이용하여 하나의 코드 구조로 관리해요.
- 동적 resolver 를 이용해서 실시간 검증도 깔끔하게 처리해요

판단 — history.state → query-param

BEFORE

수정 클릭 → 상세 API → history.state 전달 → 새로고침 시 전소실

AFTER

?id=12345678 진입 시 상세 fetch → URL에 상태 보존 → 새로고침 OK

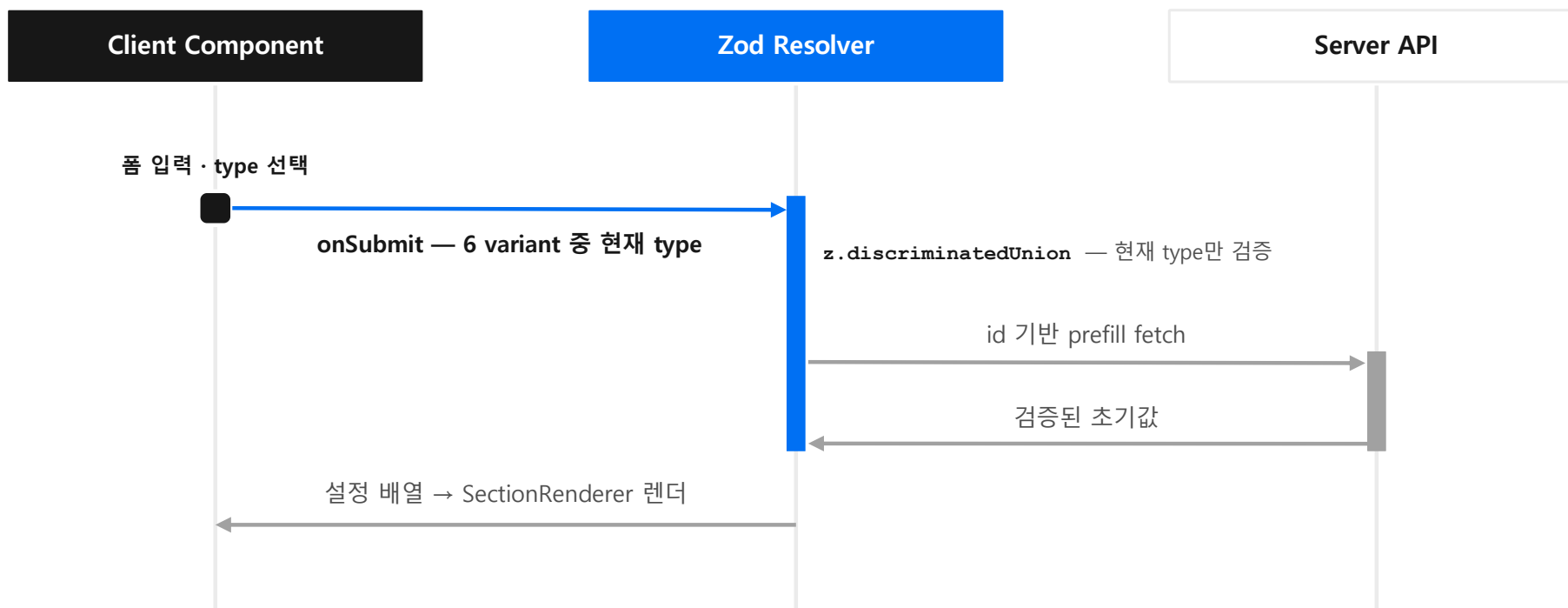
- 트레이드오프 판단
- history.state 구조는 애초에 URL 공유가 동작하지 않아요
 - 깨질 하위호환이 없음을 확인하고 전환했어요.
 - 유효하지 않은 id는 ErrorBoundary로 목록 fallback 했어요.

역량

비즈니스 이해를 기술로 연결하는 설계

거래처 관리 시스템 리뉴얼

폼을 제출하면 런타임에 실제로 이런 순서로 흘러가요



역량

비즈니스 이해를 기술로 연결하는 설계

거래처 관리 시스템 리뉴얼

협업 방식도 다시 설계했어요 — 채널마다 흩어져 있던 서버 API·디자인·기획 싱크를 한 창구로 모았어요

AS-IS

통합 테스트 문서

TO-BE

X 서버 API 스펙은 스프레드 어딘가에

X 디자인·기획 변경은 각자의 채널에

X 실시간 이슈는 물어봐야 아는 상태

확인하러 다니는 비용이 곧 개발 지연이었어요



API 단위 칸반



개발 전 싱크 점검



실시간 이슈 창구

✓ 작업 상태가 한눈에

어디가 막혔는지 바로 보여요

✓ 스펙은 개발 전에 확정

이슈가 개발 중에 터지지 않아요

✓ 소통은 한 곳으로 수렴

물어보러 다니지 않아요

예정 20일 → 15일 완료 · 25% 절감

역량

비즈니스 이해를 기술로 연결하는 설계

재무회계 N벌 → 1벌 코드 통합

9개 계열사 복불 구조를 단일 코드베이스로 — 차이는 설정 서버로 분리했어요

BEFORE



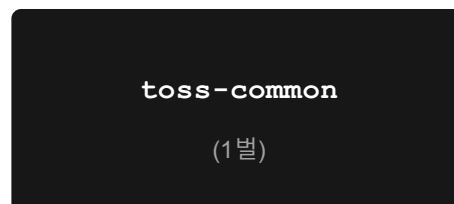
같은 코드가 계열사마다 한 벌씩 — 총 9벌



요구사항 1개 = 9곳 수정

복불 9회 · 하나만 놓쳐도 계열사 간 동작 불일치

AFTER



요구사항 1개 = 1곳 수정

설정 서버로 차이 분리

- JSON → Object 매핑으로 계열사 차이를 코드 밖으로
- 비개발자도 GUI로 계열사별 동작 조정
- 요구사항 대응이 9곳 수정 → 1곳 수정으로 — 업무 대응 속도 향상
- 어떤 계열사가 다른지 설정 파일만 봐도 파악

단기 통합 비용 < 장기 유지보수 이득이라 판단했어요

역량

비즈니스 이해를 기술로 연결하는 설계

채용 페이지 마이그레이션 (Server-Driven UI)

백오피스가 통째로 바뀌는데 기획 문서는 없었어요 — 유연한 렌더링 구조와 역기획으로 풀었어요

Server-Driven UI 렌더링 흐름



JD 타입 분기 — basic / 논포폴

- basic: 이력서 첨부 필수
- 논포폴: 첨부 선택 (단 지원 payload엔 포함)

기획 문서 부재 → 역기획으로 보완했어요

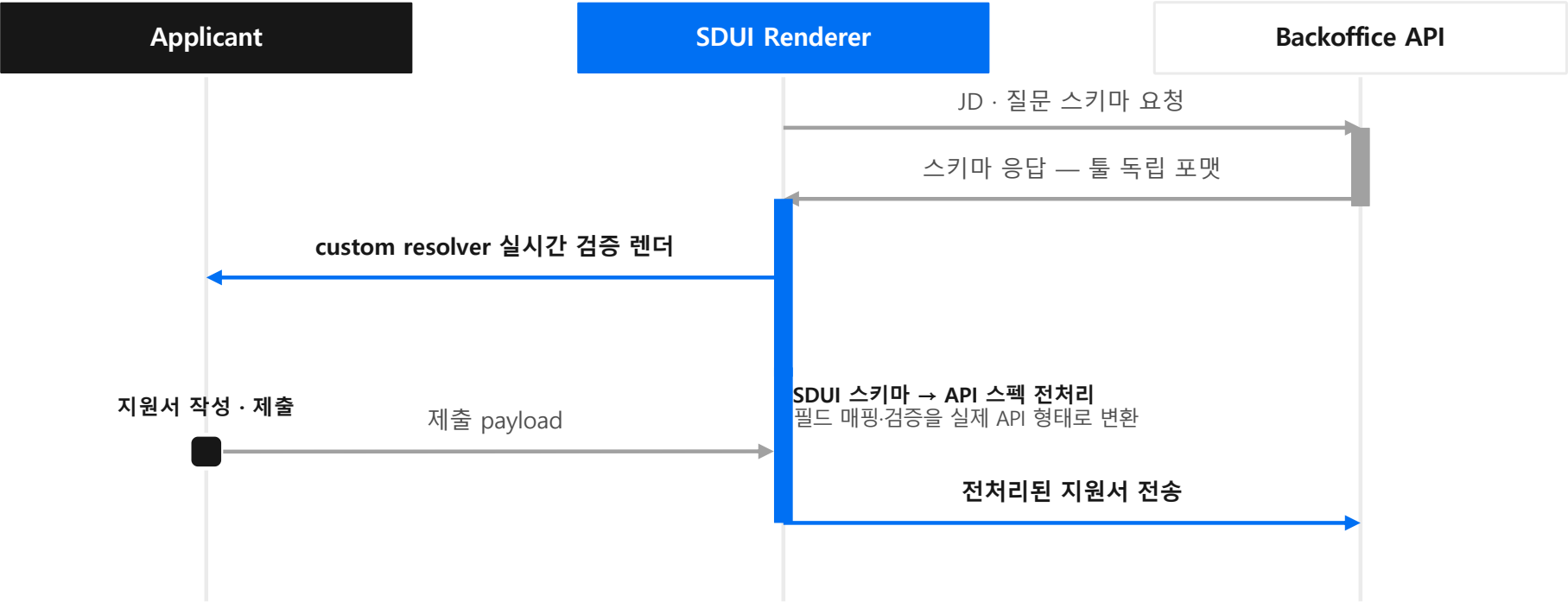
문서가 없는 상황에서 전체 유즈케이스를 역으로 구조화해 마이그레이션 누락을 막았어요. 도메인을 코드 구조로 번역한 사례예요.

역량 비즈니스 이해를 기술로 연결하는 설계

채용 페이지 마이그레이션 (Server-Driven UI)

백오피스가 바뀌어도 프론트가 그대로인 이유를 시퀀스로 보여드려요

greenhouse ⇄ workday



역량

비즈니스 이해를 기술로 연결하는 설계

검사·설문 Server-Driven UI

질문 유형도, 분기도 데이터예요 — DB 추가만으로 신규 검사가 열려요

SCHEMA - Question + Option 2테이블

question

answer_mode (13종 enum) · order · image_uri
t_question - jsonb 다국어 { ko, en }

↓ 1 : N

option

t_option · score(배점) · order
next_question_id - 선택지별 '다음 질문' = 분기 정보

GraphQL 조회 — t_question(path: \$language)로 현재 언어만 추출해요

RENDERER - answer_mode가 화면을 결정

SINGLE_CHOICE

MULTI_CHOICE

ICON_MULTI

CUSTOM_TEXT

CUSTOM_NUMBER

CUSTOM_DATE

CUSTOM_BIRTH

CUSTOM_AGE

CUSTOM_GENDER

CUSTOM_HEIGHT

CUSTOM_WEIGHT

CUSTOM_BMI

INSTRUCTION

SINGLE_CHOICE → 라디오, 선택 즉시 다음 질문으로 자동 이동

CUSTOM_* → TextInput·DatePicker (키보드 분기) · INSTRUCTION → 안내 + 시작 버튼

boolean 플래그 + 조건부 렌더링 — answer_mode별 전용 렌더 함수

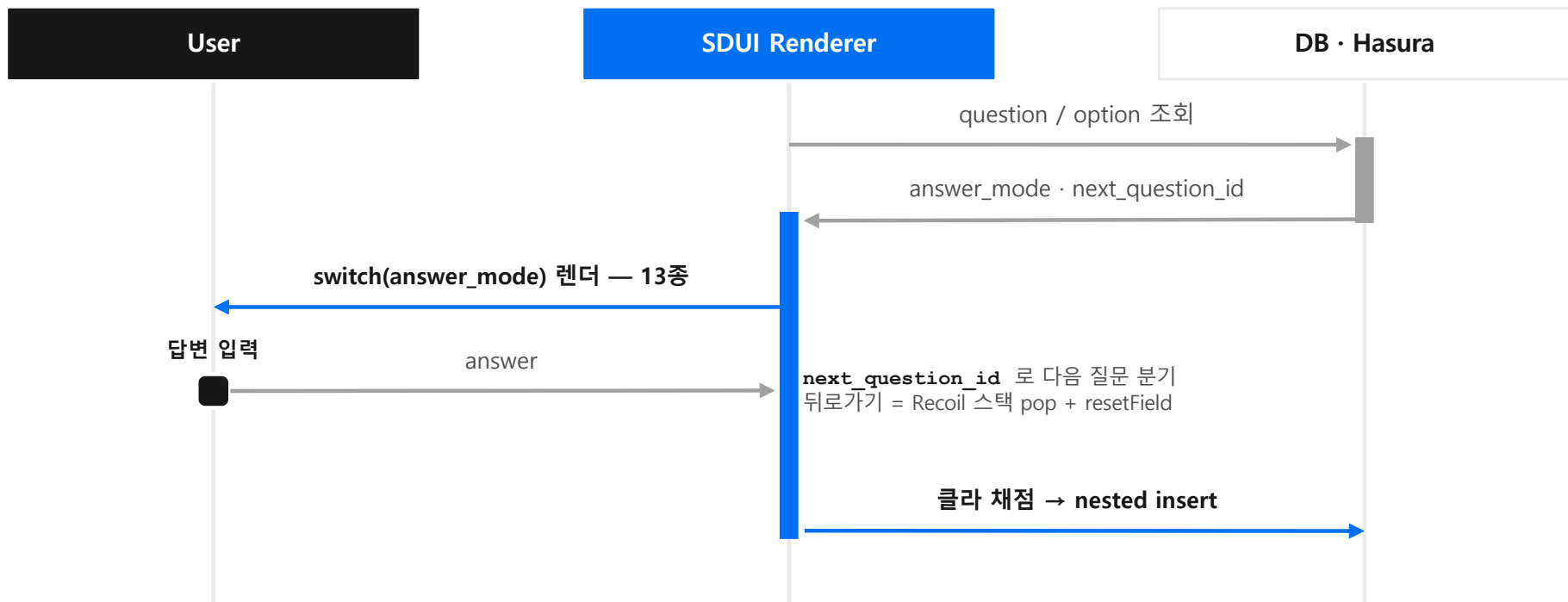
신규 검사 = DB 추가만, 프론트 리소스 0 — 일반화 범위는 “유형은 더 늘지 않는다”는 상담사 확인으로 못박았어요

역량

비즈니스 이해를 기술로 연결하는 설계

검사·설문 Server-Driven UI

답변부터 채점까지 — 데이터가 화면을 만드는 순서예요



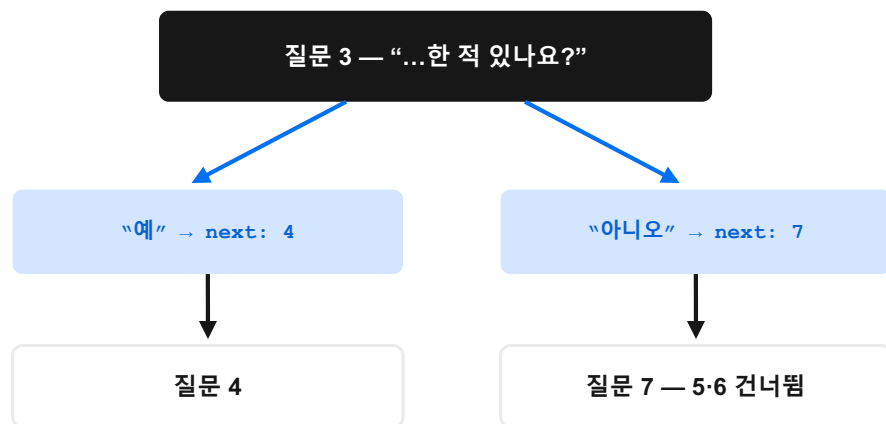
역량

비즈니스 이해를 기술로 연결하는 설계

검사·설문 Server-Driven UI

선택지의 next_question_id가 분기를, 히스토리 스택이 뒤로가기를 책임져요

분기 - 데이터로 표현, 해석은 클라이언트



스택 [1, 3, 7] — 전진은 push, 뒤로가기는 pop + 응답 초기화. 분기를 타도 역순 복귀가 정확해요

제출 - 수집부터 채점까지 클라이언트가 완결

react-hook-form 수집 { “42”: “전혀 없었다”, “43”: [...] } — 질문 ID가 필드명

정규화 질문 ID·order 매핑, 미응답 제거

클라이언트 채점 21종 검사별 전용 함수 — 역산 항목·하위 척도·BMI

GraphQL nested insert answers + scores 일괄 저장

역량

비즈니스 이해를 기술로 연결하는 설계

온보딩 튜토리얼

간편 구현 대신 확장 가능한 설계를 택했어요 — 그 판단의 근거를 보여드려요

OPTION A - 별도 화면 (간편)

- 빠르게 만들 수 있어요
- 이후 홈 화면이 바뀌면 튜토리얼과 어긋나요
- 지속적인 유지보수 부담이 생겨요

OPTION B - 기존 홈 확장 (채택)

- 별도 화면을 만들지 않았어요 — 기존 홈 인터페이스를 확장했어요
- react-native-hole-view로 실제 홈 위에 overlay를 얹었어요 — 안내 영역만 뚫어 실제 UI를 그대로 노출해요
- 홈이 바뀌면 튜토리얼에 자동 반영돼요 — 유지보수 부담이 사라져요

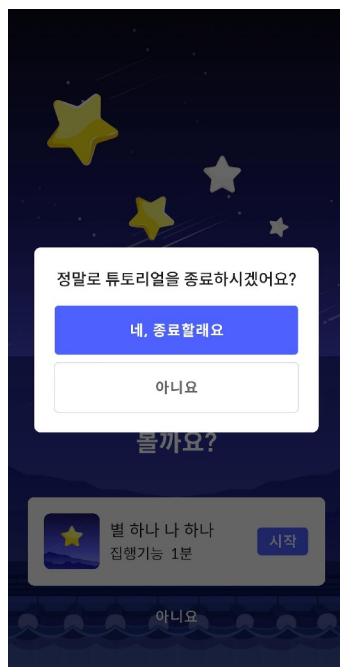
'간편 구현 vs 확장성'에서 확장성을 택했어요. 비즈니스를 먼저 이해하고 설계를 선행해야 나중에 고생하지 않아요. — 제 개발 철학이 만들어진 지점이에요.

역량

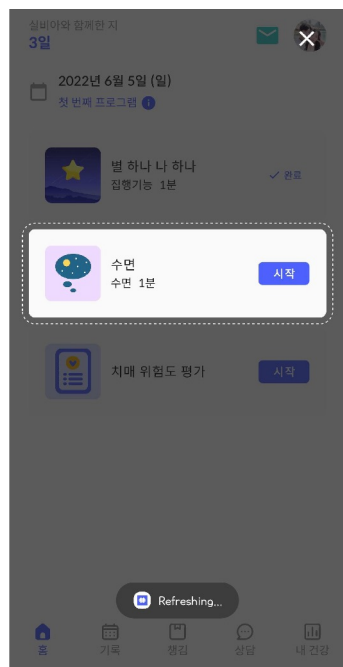
비즈니스 이해를 기술로 연결하는 설계

온보딩 튜토리얼 — 실제 화면

홈을 확장하고 overlay로 안내 — 별도 화면 없이 튜토리얼을 엮었어요



튜토리얼 안내



종료 확인

홈 위에 overlay를 엮어요

별도 튜토리얼 화면을 만들지 않고, 기존 홈 UI 위에 안내 영역만 뚫어 실제 화면을 그대로 보여줘요.

안내 영역만 하이라이트

점선으로 강조된 카드가 실제 홈의 컴포넌트예요. 홈이 바뀌면 튜토리얼도 자동으로 따라가요.

종료 흐름도 같은 홈 위에서

튜토리얼 종료 다이얼로그까지 동일한 홈 화면 위에서 처리 — 이중 화면 관리가 사라졌어요.

역량

비즈니스 이해를 기술로 연결하는 설계

전역 리팩토링

Presenter 하나가 모든 로직을 떠안고 있었어요 — Screen·Module·Component로 역할을 나눴어요

BEFORE - Container · Presenter

Container — 온갖 state를 다 선언해요



Presenter — UI와 비즈니스 로직이
한 곳에 뒤섞여요 → 비대·저가독

로직·표현·상태의 경계가 없었어요.
새 기능마다 Presenter가 계속 부풀었어요.

AFTER - Screen · Module · Component

Screen — Header·Content·Footer 골격을 그려요 · 화면 전역 혹은 주입해요

Module — 비즈니스 로직은 단일 혹은 전담해요

Component — 표현에만 집중해요 · props drilling은 전역 상태로 제거

회고 — DefaultScreen 같은 상위 추상화 여지가 있었어요. 제 설계를 스스로 비판적으로 돌아봐요.